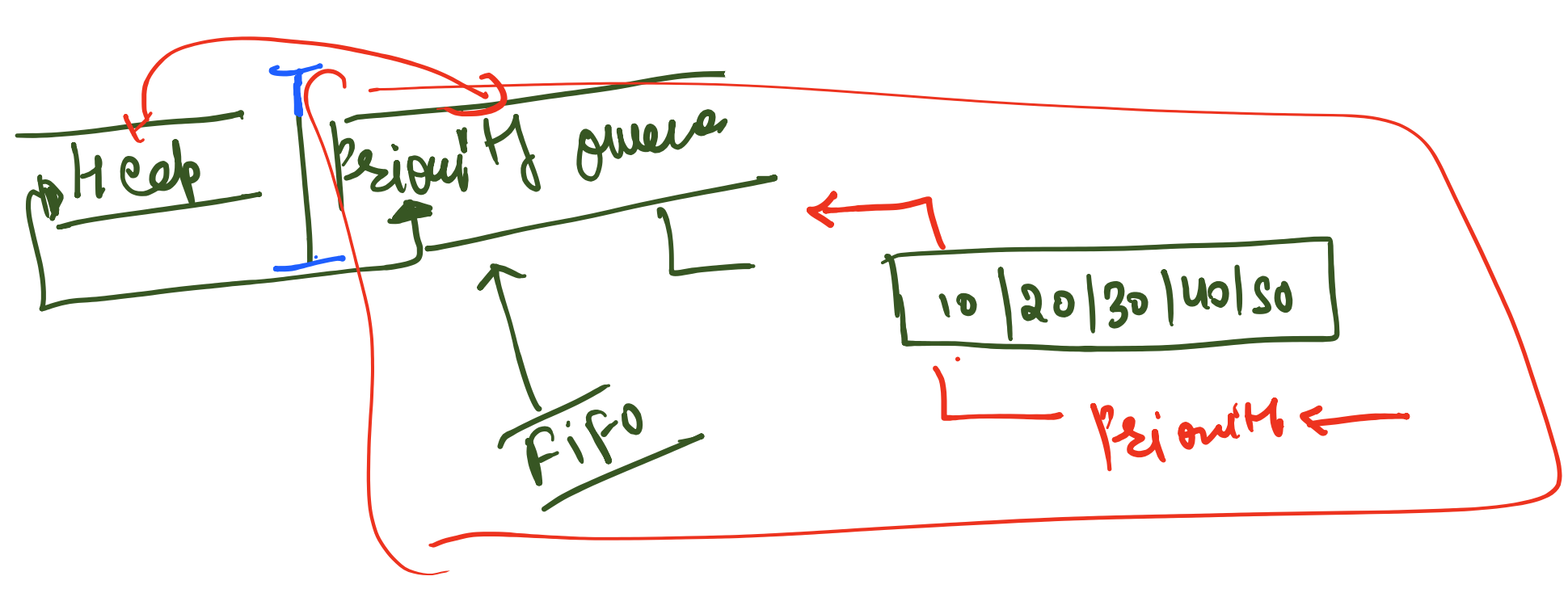
Lec-41

18/08/26 7:10 PM

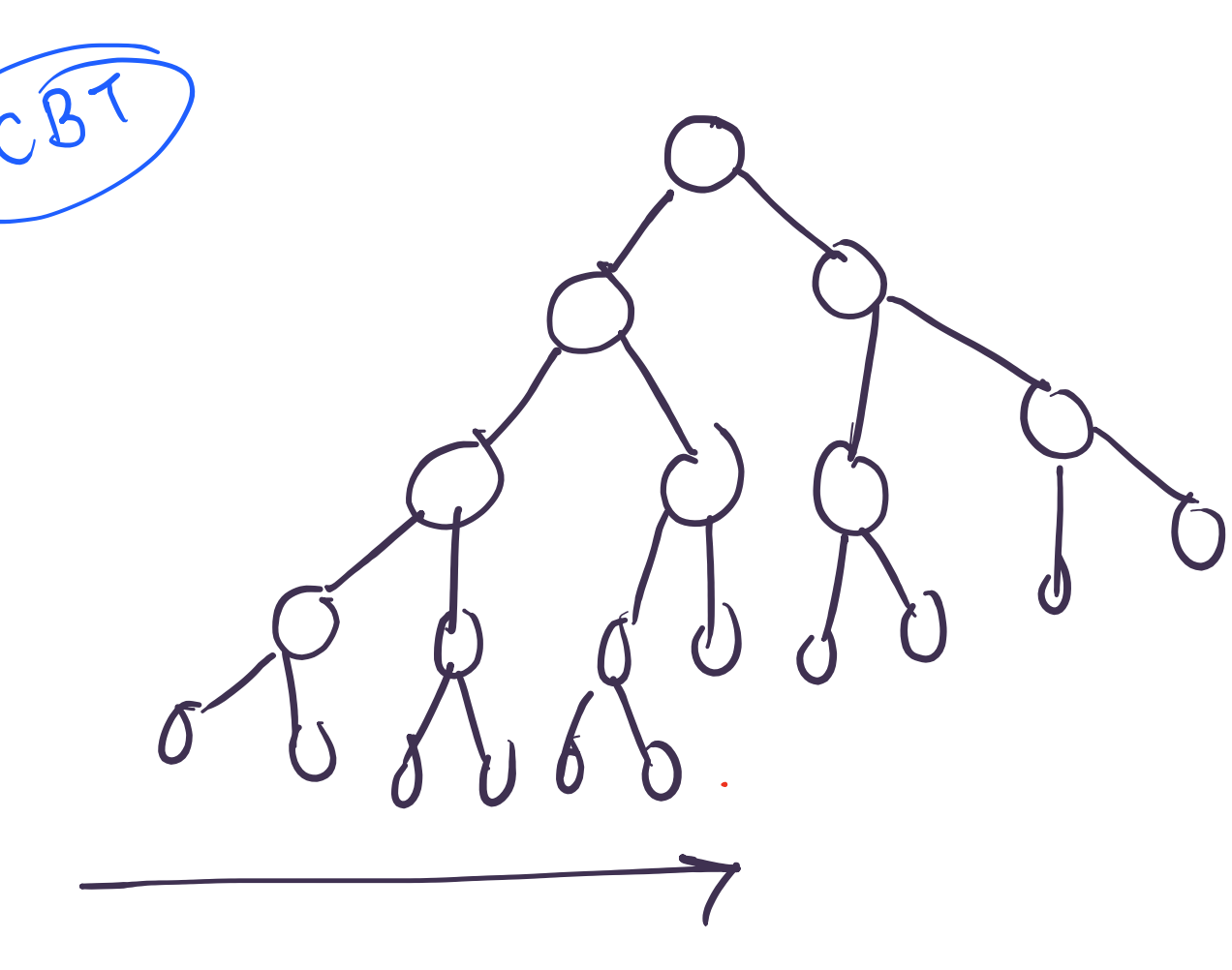


(ii) Heap

→ It is complete Binary Tree and Priority associated

CBT

CBT

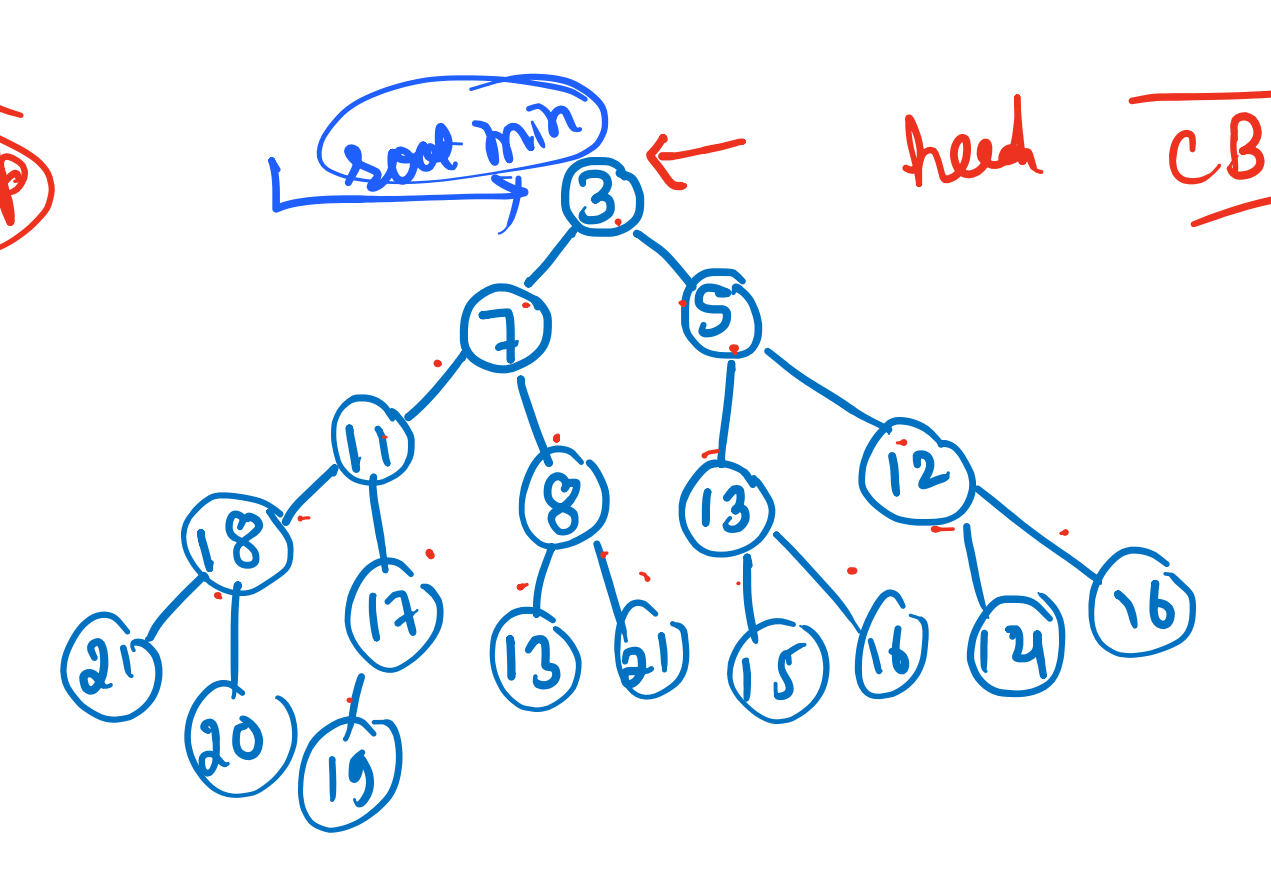


(i) min heap ← (i) Rank → Smaller has least priority

(ii) marks → max heap → [larger value has lesser priority]

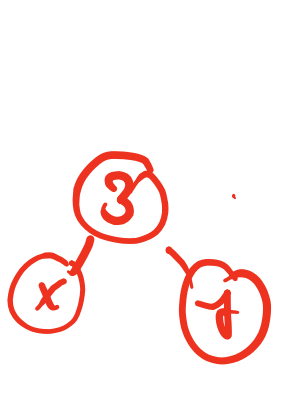
(ii) min heap

root min ← heap CBT

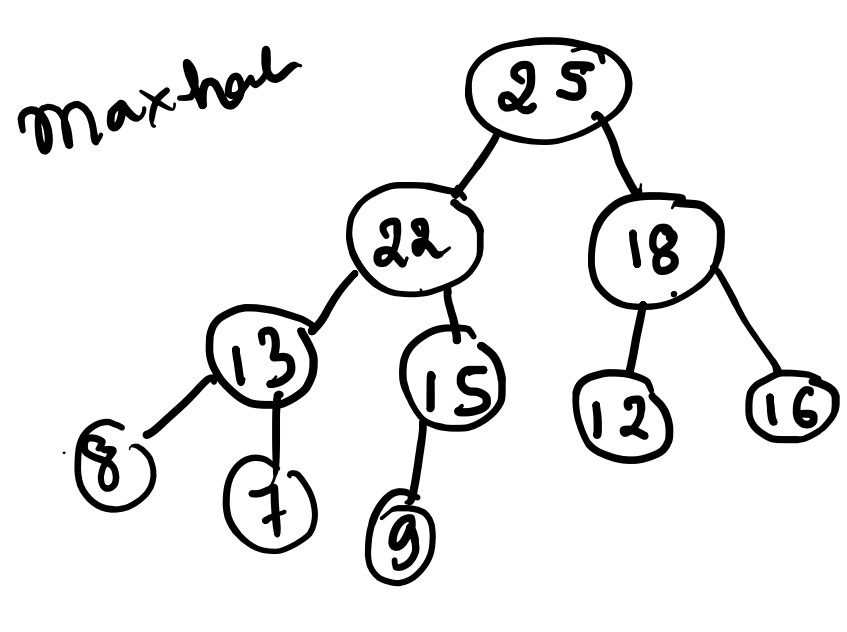


parent > child

lower value ↑

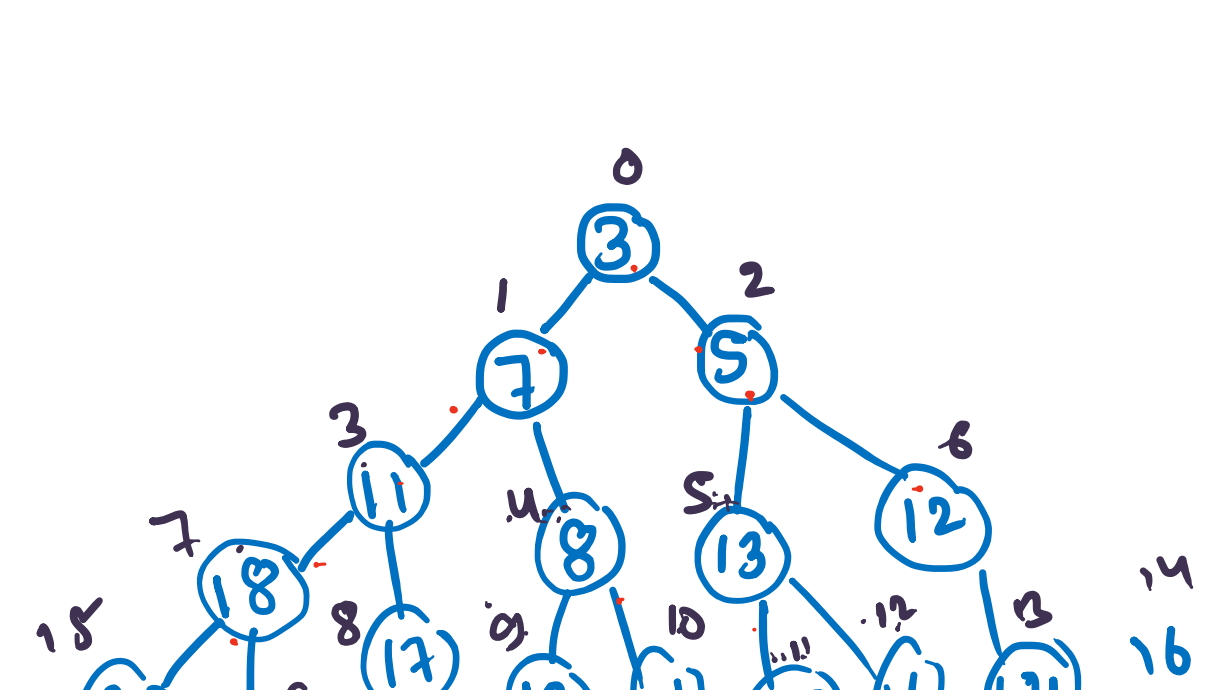


max-heap



Why Heap?

	Sorted Array	Unsorted Array	heap	
Add	O(n)	O(1)	log(n)	→ code
min get	O(1)	O(n)	O(1)	→
remove min	O(n)	O(n)	log(n)	→



LCi = 2×pi+1

RCi = 2×pi+2

pi = (ci-1)/2

12-1/2 = 5

11-1/2 = 5

4-1/2 = 1

ll.add(x)

0	1	2	3	4	5	6
1	4	2	7	5	9	3

3-1/2 = 1

1-1/2 = 0

4-1/2 = 1

1-1/2 = 0

5-1/2 = 2

2-1/2 = 0

0-1/2 = 0

2-1/2 = 0

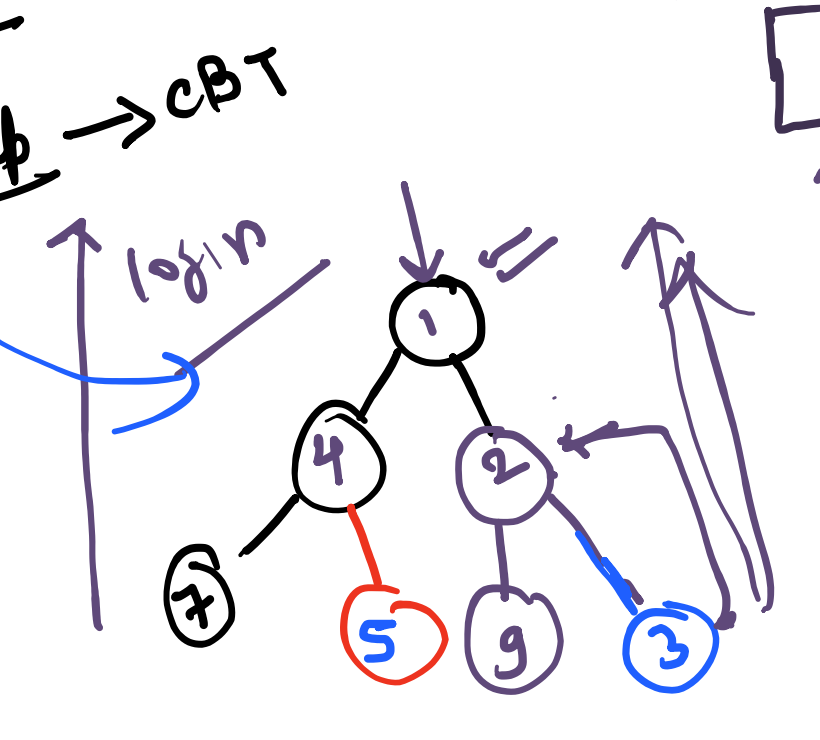
0-1/2 = 0

5-1/2 = 2

min heap → CBT

0	1	2	3	4	5
5	7	9	4	2	3

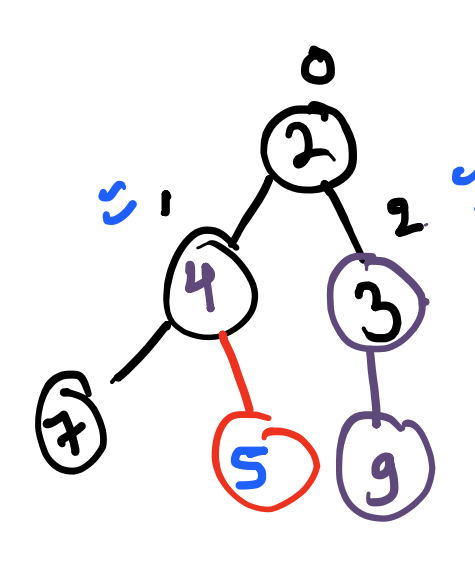
log n



p(ci)

```
private void upheapify(int ci) {
        // TODO Auto-generated method stub
        int pi = (ci - 1) / 2;
        if (ll.get(pi) > ll.get(ci)) {
            swap(pi, ci);
            upheapify(pi);

    }
```

log(n)



0	1	2	3	4	5	6
2	4	3	7	5	9	1

mini = pi 2

lci = 1

Rci = 2

2×2+1 = 5

2×2+2 = 6

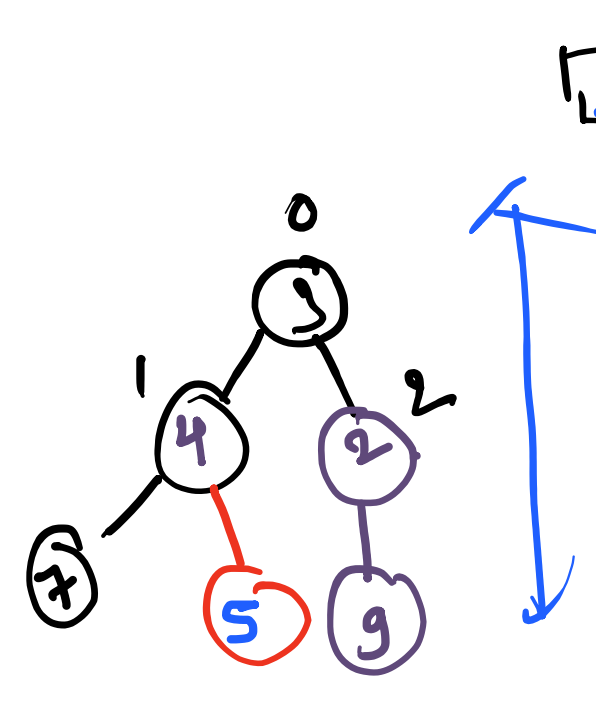
```
private void downheapify(int pi) {
        // TODO Auto-generated method stub
        int lci = 2 * pi + 1;
        int rci = 2 * pi + 2;
        int mini = pi;
        if (ll.get(lci) < ll.get(mini)) {
            mini = lci;

        if (ll.get(rci) < ll.get(mini)) {
            mini = rci;

    }
```

9 < 3

if (mini != pi)

swap (mini, pi)

0	1	2	3	4	5
2	4	3	7	5	9



```
private void downheapify(int pi) {
        // TODO Auto-generated method stub
        int lci = 2 * pi + 1;
        int rci = 2 * pi + 2;
        int mini = pi;
        if (lci < ll.size() && ll.get(lci) < ll.get(mini)) {
            mini = lci;
        }
        if (rci < ll.size() && ll.get(rci) < ll.get(mini)) {
            mini = rci;
        }
        if (mini != pi) {
            swap(mini, pi);
            downheapify(mini);
        }
}
```

→ 5

→ 6

9 < 3

Example 2:

```
Input: nums = [3,2,3,1,2,4,5,5,6], k = 4
Output: 4
```

Sort n log(n)

k

O(n) O(1)

N log(N)

N=9

K=4

K=4

[3, 2, 1, 2, 4, 5, 5, 6, 1, 2, 3]

1 4

O(K)

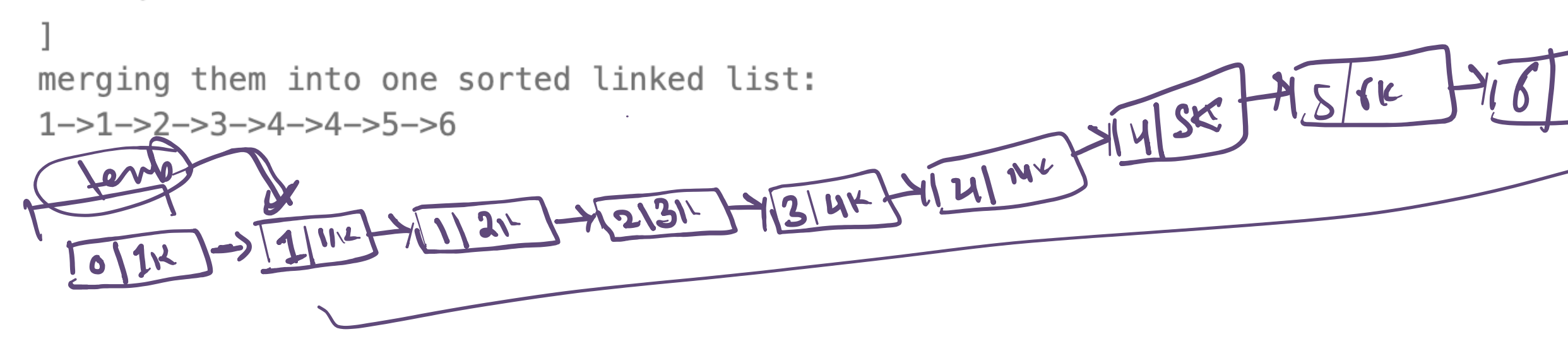
N×log(K)

6545 → min → 4

```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
Explanation: The linked-lists are:
[
  1->4->5,
  1->3->4,
  2->6
]
merging them into one sorted linked list:
1->1->2->3->4->4->5->6
```

4 8 6 → minheap

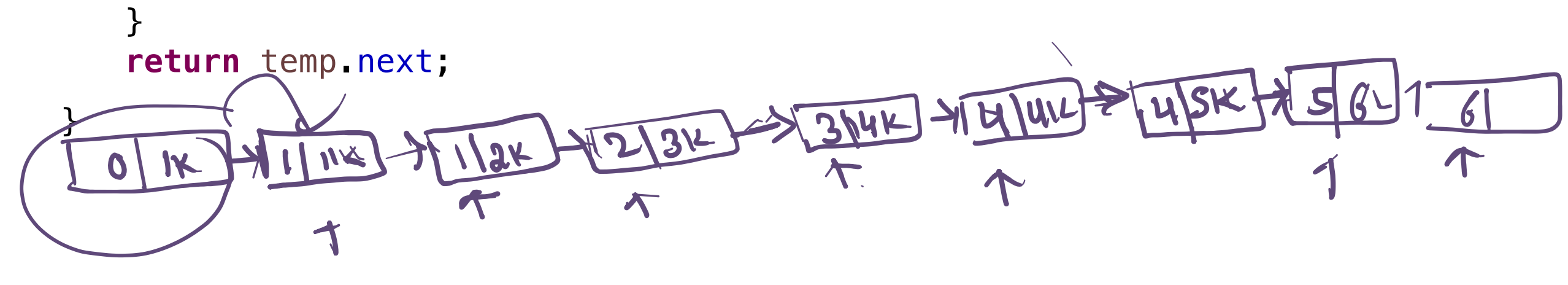
nK log(K)



```
public ListNode mergeKLists(ListNode[] lists) {
        PriorityQueue<ListNode> pq = new PriorityQueue<>();
        for (int i = 0; i < lists.length; i++) {
            if (lists[i] != null) {
                pq.add(lists[i]);
            }
        }
        ListNode dummy = new ListNode();
        ListNode temp = dummy;
        while (!pq.isEmpty()) {
            ListNode rn = pq.poll();
            dummy.next = rn;
            dummy =dummy.next;
            if (rn.next != null) {
                pq.add(rn.next);
            }
        }
        return temp.next;
```

4 8 6

```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
Explanation: The linked-lists are:
[
  1->4->5,
  1->3->4,
  2->6
]
merging them into one sorted linked list:
1->1->2->3->4->4->5->6
```



[10, 30]

[5, 10]

[15, 20]

[2, 6]

[12, 18]

[19, 25]

[1, 5]

[3, 5]

0	1
10	30
5	10
15	20
2	6
12	18
19	25
1	5
3	5

→ Algorithm (3)

1	5
2	6
3	5
5	10
10	30
12	18
15	20
19	25

10-30

5-10

3-5

15-20

2-6

12-18

19-25

1-5

→ arr[0] [1,5]

[1, 8]

Priority

~~[1, 5]~~

~~[2, 6]~~

~~[3, 5]~~

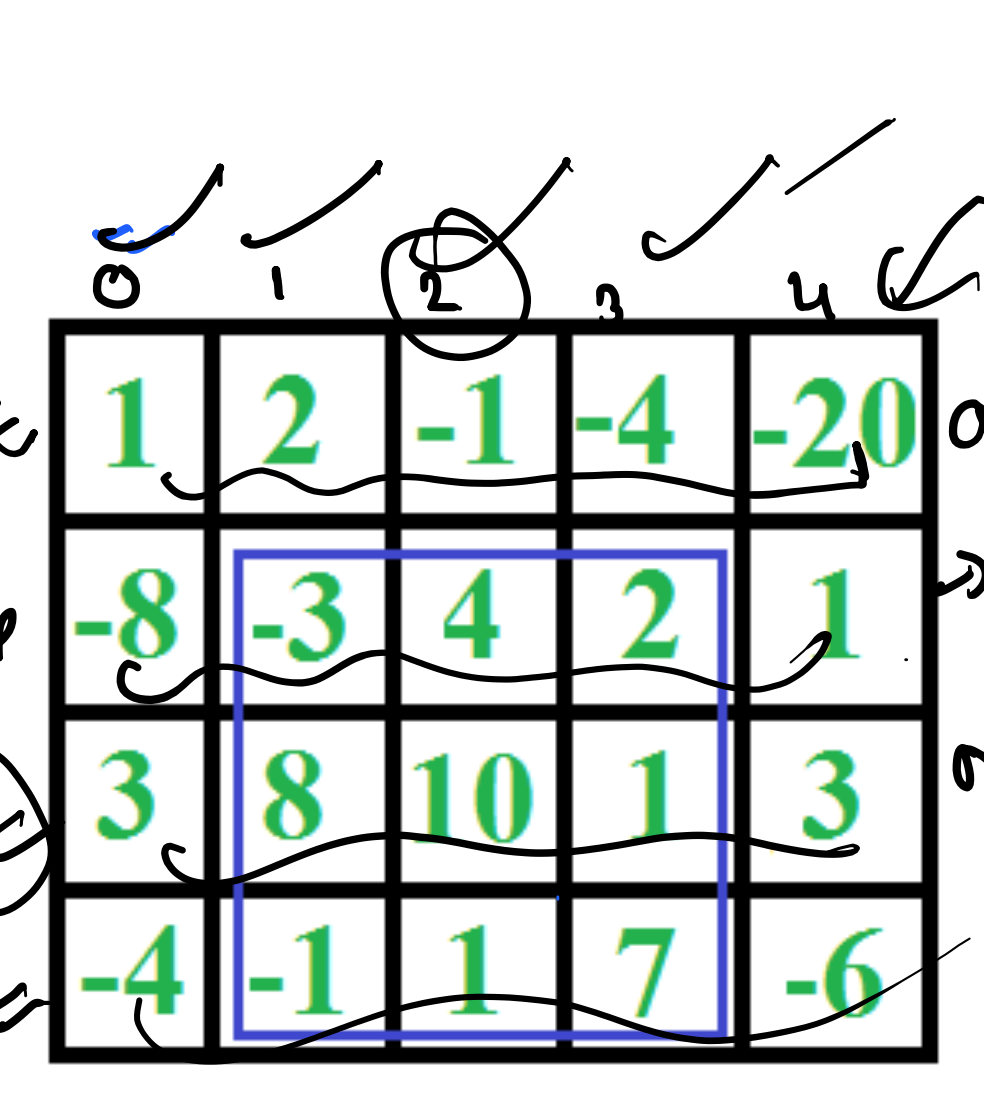
~~[5, 10]~~

[10, 30]

~~12, 18~~

15, 20

19, 25



	0	1	2	3	4
0	1	2	-1	-4	-20
1	-8	-3	4	2	1
2	3	8	10	1	3
3	-4	-1	1	7	-6

arr(0) -8, -3, 4, 2, 1

arr(1) -4 -1 1 7 -6

arr(2) 1 2 -1 -4 -20

3 8 10 1 3

1 2 -1 -4 -20

-4 -1 1 7 -6

-8 -3 4 2 1

3 8 10 1 3

```
public static int Meeting_Room(int [][]intervals) {
        Arrays.sort(intervals,(a,b)->a[0]-b[0]);
        PriorityQueue<int[]> pq = new PriorityQueue<>((a,b)->a[1]-b[1]);
        pq.add(intervals[0]);
        for (int i = 1; i < intervals.length; i++) {
            if(intervals[i][0]>=pq.peek()[1]) {
                pq.remove();
            }
            pq.add(intervals[i]);
        }
    }
```

(2) >= 5

2 >= 5

1	5
2	6
3	5
5	10
10	30
12	18
15	20
19	25

~~[1,5]~~

~~2,6~~

~~3,5~~

~~5,10~~

10,30 (15,4)

~~12,18~~

15,20